1. Layered Architecture
    a. Example: Amazon. There is a presentation layer (the UI), business layer (the calculations), persistence layer (the translation from business to database), and the database layer (the stored data).
    b. This architecture is appropriate because several engineers are able to independently work on each layer without having to worry about the other ones. With dedicated focus on one part, each layer is prioritized concurrently, maximizing efficiency.
2. Repository Architecture
    a. Example: Air traffic control. There is a central repository that connects to all components (radar, collision warning, flight plan, controller display).
    b. All components need each other at the exact same time, so the central repository sits as the hub for each of these components. Each component is able to refer to the others instantaneously.
3. Client-Server Architecture
    a. Example: Banking system. Clients (such as an ATM, mobile app, or web browser) can access the bank's server network via the internet. The bank server then accesses the data from different servers to send and retrieve the clients' data.
    b. Several clients need access to the bank's server network, and this is a safe, secure way to do so.
4. Pipe and Filter Architecture
    a. Example: E-commerce platform. Raw receipts are put into a filter (data parser and validator) which funnels into two pipes in parallel (1 is currency & tax converter and 2 is anomaly & fraud detector). These pipes then reconverge into another filter (data unparser or aggregator), which the data is then stored into the business database.
    b. With this architecture, the step-by-step nature of data processing is clear. Furthermore, there is more scalability through parallel processing and more flexibility as business rules change.

Example 1 combining multiple patterns:
Netflix - Netflix combines the Client-Server, Layered, and Repository Architectures. It uses the Client-Server when the user requests over the network to stream services via their device. It uses the Layered by following the Presentation - Business - Persistence - Database structure when the request hits the server. Inside the Persistence layer, there is a Repository sub-system, where the repository is responsible for managing movie data. This Repository sub-system decides to pull data either from local memory cache or from the database.

Strengths: There is optimal user performance because the setup allows for the fastest response from ultra-fast cache memory. This also allows for multiple devices with just one backend server. Although this includes a Repository Architecture, teams are still able to work independently.

Limitations: The system is complex, and debugging a single issue becomes significantly harder. Furthermore, a single network request must go through layers of architecture, where code is converted, parsed, and manipulated from the client to the database. This adds minor CPU strain each time a transaction occurs.